LBRO: A Law-Aware Breach Response Orchestrator with Signed Machine-Learning Triage on CICIDS2017

Veerendhra Nuthalapati
Dept. of Computer Science and Engineering
Manipal Institute of Technology, India

Yeramaneni Kedarnath Chowdary
Dept. of Computer Science and Engineering
Manipal Institute of Technology, India

Daasharathi Darahaas Yandam
Dept. of Computer Science and Engineering
Manipal Institute of Technology, India

Kummarasani Varun
Dept. of Computer Science and Engineering
Manipal Institute of Technology, India

Corresponding author email: to be added

***Abstract*—Small organisations often lack the staff and tooling to detect intrusions, preserve evidence and meet breach-notification deadlines. We present LBRO (Law-aware Breach Response Orchestrator), an open platform that ingests security events over an authenticated REST API, classifies network-flow events with a machine-learning model, opens and tracks incidents, stores evidence with SHA-256 integrity checks and chain-of-custody records, and records notification obligations for each incident. The classifier is a histogram-based gradient-boosted tree trained on all 2,830,743 flows of the CICIDS2017 MachineLearningCVE release, using 77 flow features and 15 traffic labels. On a stratified 20% held-out test set it reaches accuracy 0.9983 and macro F1 0.8773; five-fold cross-validation gives macro F1 0.8485 $\pm$ 0.021. Flood, scan and password brute-force attacks are detected with F1 above 0.97, whereas web attacks, Infiltration and Bot are weak. The model ships as a signed bundle whose file hashes and HMAC-SHA256 manifest signature are verified before deserialisation. We describe the architecture, the evaluation protocol and its known weaknesses, the security controls, and a verification pass over the released model bundle. We argue that reporting rare-class weakness openly matters most for triage tools that feed human analysts.**

***Index Terms*—intrusion detection, incident response, gradient boosting, CICIDS2017, evidence integrity, breach notification, model supply chain**

I. Introduction

Incident response depends on three capabilities that many small and medium organisations struggle to combine: continuous detection, defensible evidence handling, and a clear record of obligations under breach-notification law. Such laws include the European Union's General Data Protection Regulation (GDPR), the United States' Health Insurance Portability and Accountability Act (HIPAA) breach rules, and India's Digital Personal Data Protection Act (DPDPA). Commercial security information and event management (SIEM) and orchestration products address these needs, but at a cost and complexity that many teams cannot absorb. Open tooling tends to cover one function, which leaves analysts copying data between systems and reconstructing timelines by hand.

LBRO addresses this gap in a single deployable system. Applications send events to an authenticated endpoint. Events that carry network-flow features are scored by a classifier, and high or critical events open incidents. Each incident collects evidence files with cryptographic hashes and a custody log, records the machine-learning decision with its model version, and carries data-classification flags that determine which notification frameworks may apply.

This paper makes four contributions. First, it describes an end-to-end architecture that links detection, evidence and compliance through one incident data model. Second, it reports an offline evaluation of a gradient-boosted classifier on the full CICIDS2017 release, including five-fold cross-validation and per-class results. Third, it documents a model-distribution scheme in which the classifier cannot be deserialised unless its artifacts match a signed manifest, which treats the model as part of the software supply chain. Fourth, it reports limitations in detail, including weak web-attack detection, a model-selection bias we found in our own tooling, and the gap between offline scores and operational use.

II. Related Work

A. Machine Learning for Intrusion Detection

Buczak and Guven surveyed data-mining and machine-learning methods for intrusion detection and identified the tension between detection accuracy and deployment cost as the central practical problem [1]. Sharafaldin, Lashkari and Ghorbani released CICIDS2017, a labelled capture of benign and attack traffic whose flows are summarised by 80 features produced with CICFlowMeter [2]. Work on this dataset often reports high accuracy for ensemble methods. Because the benign class dominates the data, accuracy alone flatters a classifier that misses rare attacks, so macro-averaged F1 and per-class reporting are more informative.

Sommer and Paxson argued that machine learning for network intrusion detection is difficult to use operationally, because of the cost of false alarms, the difficulty of collecting

TABLE I
TYPICAL COVERAGE OF TOOL CATEGORIES COMPARED WITH LBRO.

Capability	SIEM/ SOAR	Forensic tools	Compliance trackers	LBRO
Ingestion and detection	Yes	Partial	No	Yes
Evidence hashing and custody	Partial	Yes	No	Yes
Notification deadlines	No	No	Yes	Yes
Verified model bundles	Varies	No	No	Yes
Open, self-hostable	Varies	Varies	Varies	Yes

representative training data, and the scarcity of feedback from analysts [3]. LBRO follows their recommendation to route uncertain predictions to people rather than act on them automatically. Histogram-based gradient-boosted trees are available in scikit-learn [4], and comparable boosting systems such as XGBoost [5] and LightGBM [6] belong to the same design family.

B. Incident Response, Forensics and Compliance

NIST's incident-handling guidance structures response into preparation, detection and analysis, containment and recovery, and post-incident activity [7]. NIST's forensic integration guidance describes collection, examination, analysis and reporting of digital evidence [8]. Both assume that analysts can rely on the integrity of the evidence they handle. Forensic tools preserve artifacts but seldom track breach-notification deadlines, while compliance tracking systems record deadlines but do not ingest events. LBRO's contribution is the coupling of these functions in one data model, not a new detection algorithm. Table I summarises the coverage of each tool category in general terms; it describes typical products, not a survey of specific vendors.

III. System Architecture

The system architecture (Fig. 1) has four components: a React 18 single-page application served by nginx, which proxies `/api/` requests to a FastAPI service; PostgreSQL 16, accessed through SQLAlchemy 2.0 and asyncpg; and a separate SQS worker. Asynchronous work, including classification of incidents created without inline features and notification retries, is placed on Amazon SQS queues and processed by the worker. Deployment is covered in Section VII.

A. Authentication and Authorisation

Applications authenticate to the ingestion endpoint with a project-scoped Bearer key, which is stored as a hash. Human users authenticate with an email address and password and receive a JSON Web Token (JWT) signed with HS256 and valid for 30 minutes. Logout revokes the token identifier, and the revocation is checked on protected endpoints; an integration test confirms that a revoked token is rejected by the `/me` endpoint. After five consecutive failed logins an account is locked. Role-based access control separates a platform superadministrator from project-level admin, analyst and viewer roles. Project-isolation tests confirm that one project's data is not visible to another project's members. Table II lists each control with the tests that exercise it.

B. Event Ingestion and Incident Creation

An event is stored with a processing status and one of five severities: critical, high, medium, low or info. If its payload contains network-flow features, the classifier runs inline; otherwise a heuristic maps the event type to an attack category. Heuristic results are always marked for review. Events of severity high or critical create an incident. The model changes an event's severity only when its prediction is confident and not flagged for review, so an uncertain automated label cannot raise the urgency of the event.

C. Incident Presentation

The incident stores the sender's title and severity together with the model's category, confidence, model version and review flag. These two sources can disagree. An early version of the overview showed the sender's title and the model's verdict without comment, which produced a confusing page in which a high-severity web-attack title sat beside a BENIGN classification. The overview now shows a warning whenever the reported severity is high or critical and the model says BENIGN, stating both values and the model's confidence. The explanation for a BENIGN verdict also omits the source address, port and duration, because those describe an attack that did not happen. Explanations are fixed templates per category rather than generated text, which keeps every statement reviewable.

D. Evidence Store

Evidence is stored in PostgreSQL with its filename, content type, size and SHA-256 hash. Automatically captured ingestion evidence is marked immutable and receives a chain-of-custody entry with the action *ingested* and a note naming the endpoint that produced it. The verify endpoint recomputes the hash over the stored bytes and compares it with the recorded value. It returns both a boolean result and the computed digest, so the interface can show the outcome from the server's computation. Downloading evidence requires a dedicated permission, and audit records include the acting user and the client IP address.

E. Compliance Obligations

Each incident carries data-classification flags for personal, health and other sensitive data. The compliance module derives which notification frameworks may apply from those flags and the incident's jurisdiction. The rule set is deterministic and inspectable, so results are reproducible. It has not been reviewed by counsel, and the platform presents these outputs as a workflow aid rather than as legal advice.

IV. Worked Example: From Event to Evidence

To make the data flow concrete, consider an event that a developer sent during integration testing. The payload described a web attack with a high severity and a loopback source address, and it carried a full set of network-flow features. The

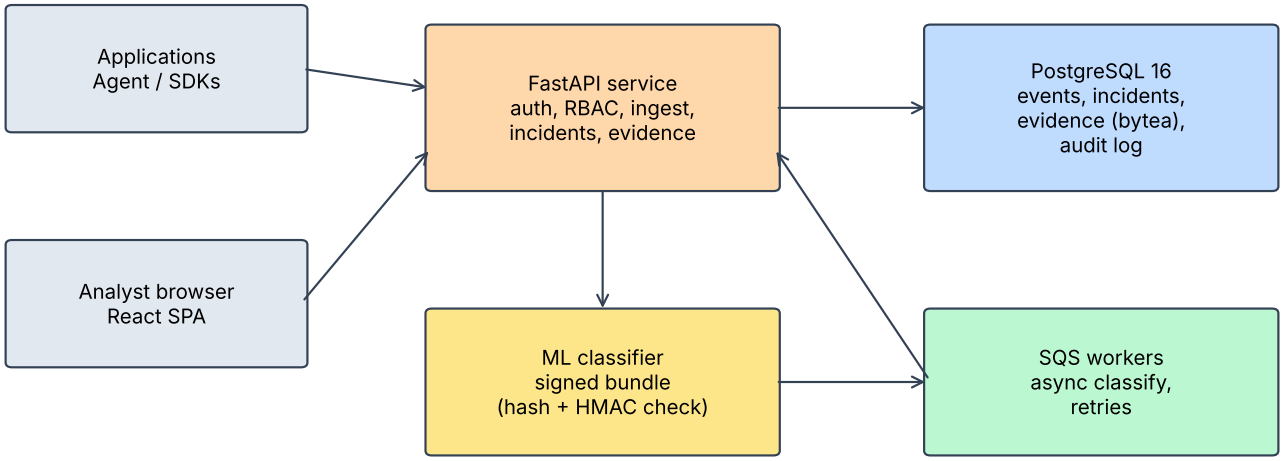


Fig. 1. LBRO system architecture. Applications send events to the API; classification and retries run on SQS workers; PostgreSQL holds events, incidents, evidence and the audit log.

TABLE II
SECURITY CONTROLS AND THE TESTS THAT EXERCISE THEM.

Control	Mechanism	Test evidence
Ingestion key storage	Keys stored as hashes	`test_api_key_security.py`
Login lockout	Lock after five failures	`test_account_lockout_after_five_failures`
Token revocation	Revoked `jti` rejected	`test_revoked_token_rejected_on_me_endpoint` (integration)
Project isolation	Per-project data access	`test_project_isolation.py`
Role-based access	Permission matrix by role	`test_rbac.py`
Model integrity	File hashes and HMAC signature	`test_tampered_model_rejected`
Evidence integrity	SHA-256 recomputed on verify	`test_sha256_verify_endpoint`
Auto-containment gate	Off by default; all conditions required	Seven `test_auto_contain_blocked_when_*` cases and a default-off test

request was accepted and stored with a pending status, the classifier ran inline, and the model returned BENIGN with 95% confidence. Because the reported severity was high, an incident was created. The incident therefore carried a high severity and a title naming a web attack from the sender, together with a benign verdict and a confidence from the model.

The overview displayed both values and the disagreement warning. The explanation shown for the BENIGN verdict describes likely normal application activity and omits the source address, as a benign verdict requires. Two evidence items were captured automatically: a network-flow JSON document and an ingestion-event JSON document. Each has its own SHA-256 digest and an *ingested* chain-of-custody entry naming the endpoint. Verification recomputes the digest over the stored bytes, so an analyst can confirm that the record matches what was ingested. This example also exposed two presentation defects, a verification badge that reported a mismatch for matching hashes and a mislabelled code sample, which we corrected before writing this paper.

V. METHODOLOGY

A. Dataset and Preprocessing

We used the eight daily MachineLearningCVE files of CICIDS2017, containing 2,830,743 flows with 77 numeric features and 15 labels. Of these, 2,273,097 flows are benign and 557,646 are attacks. The raw labels contain a mojibake form of the web-attack names, which we normalised to *Web Attack – Brute Force*, *Web Attack – XSS* and *Web Attack – Sql Injection*, keeping the dataset's spelling of the last name. Some column names differ between the daily files and are aligned through an explicit alias table, so every file maps to the canonical feature order defined in the application's feature module. The manifest records the SHA-256 digest of each source file, which identifies the training data exactly.

B. Model

The classifier is a `HistGradientBoostingClassifier` with 100 boosting iterations, balanced class weights and

random state 42. Missing values are replaced by per-feature medians computed on the training split. Out-of-distribution bounds, also learned from the training split, flag feature values outside the observed range. The predicted class is the index of the highest class probability, mapped to a name through the stored class list. That list is the sorted order of the 15 class names.

C. Model Selection

Four candidates were compared on a stratified sample of 119,616 flows: Gaussian Naive Bayes, a decision tree, a random forest and histogram gradient boosting. Naive Bayes was not viable, at macro F1 0.440. Table III reports the other three. Gradient boosting had the highest five-fold cross-validated mean and was selected. The final model was then trained on all 2.83 million flows. The decision tree and random forest were not retrained on the full data.

We must report a bias in our own tooling. An earlier selection script ranked candidates by a composite score computed on the test split, which would make the selected model's test score optimistic. The comparison in Table III uses the sample's own split and cross-validation. Because the selection history includes that test-based ranking, the table is best read as indicative rather than as a clean benchmark.

D. Evaluation Protocol

We report accuracy, macro F1 (the unweighted mean of per-class F1), weighted F1 and per-class F1. Macro F1 is the primary metric because the benign class makes accuracy uninformative. The final test uses a stratified 20% hold-out with random state 42. Stability is measured with five-fold cross-validation, reporting the mean and standard deviation of the fold scores. Each fold uses one training run, so the reported spread reflects variation across folds rather than across random seeds.

Two caveats apply to the cross-validation. Its median-imputation values were computed on the full dataset rather than on each training fold, a small form of leakage that slightly favours the reported mean. It was also run on scikit-learn 1.9.1, whereas the shipped bundle records 1.7.2, the version pinned for the application. We report both caveats because they affect how the cross-validated figures should be compared with the held-out figures.

E. Signed Model Bundle

A bundle contains five files: the serialised model, its preprocessing parameters, the class list, a manifest and a signature. The manifest records the SHA-256 digest of each artifact, the digests of the training files, the library versions and the metrics. The signature is an HMAC-SHA256 over the manifest bytes, computed with a deployment secret that is kept out of the bundle. At load time the loader verifies every file digest and the signature before calling the deserialiser. It refuses unsigned bundles in production and refuses a bundle whose scikit-learn major or minor version differs from the runtime. Because pickle-based deserialisation can execute code, this ordering is the main barrier between a tampered model file and code execution on the API host [9].

TABLE III
MODEL SELECTION ON A 119,616-FLOW SAMPLE (MACRO F1).

Model	Single split	5-fold CV	Decision
Histogram gradient boosting	0.913	0.908 ± 0.035	Selected
Decision tree	0.916	0.898 ± 0.022	Alternative
Random forest	0.885	0.890 ± 0.013	Not selected
Gaussian Naive Bayes	0.440	not run	Not viable

TABLE IV
PER-CLASS F1 ON THE 20% HOLD-OUT.

Class	F1
DDoS, FTP-Patator, Heartbleed	1.000
DoS Hulk, PortScan, SSH-Patator, BENIGN	0.997–0.999
DoS GoldenEye	0.988
DoS Slowhttptest	0.984
DoS slowloris	0.978
Infiltration	0.778
Bot	0.774
Web Attack – Brute Force	0.717
Web Attack – XSS	0.504
Web Attack – Sql Injection	0.444

VI. RESULTS

A. Experimental Setup

All training and evaluation used the eight CICIDS2017 daily files listed in the manifest by SHA-256 digest. The training pipeline ran on Python 3.12 with NumPy 1.26.4, joblib 1.4.2 and the scikit-learn version recorded in each run's manifest. The shipped bundle was produced with scikit-learn 1.7.2. Random state 42 governs the hold-out split and the classifier. Each reported figure comes from one run; we did not measure variation across seeds, so the fold standard deviations are the only estimate of spread. Scores are computed in double precision over the full test partition, with no subsampling for the final model.

B. Held-out and Cross-Validated Performance

On the 20% hold-out the model reaches accuracy 0.9983, weighted F1 0.9983 and macro F1 0.8773. Five-fold cross-validation gives accuracy 0.9975 and macro F1 0.8485 with a standard deviation of 0.021. The gap between the two macro F1 values comes mainly from the small classes, which fluctuate across folds.

C. Per-Class Behaviour

Table IV gives held-out per-class F1, and Fig. 2 plots the same values. Floods, port scans and password brute-force attacks on FTP and SSH are detected almost perfectly. The weak classes are the three web attacks, Infiltration and Bot. Several rare classes have very few test flows, so a score of 1.000 for Heartbleed should not be read as a reliable estimate.

Web attacks are difficult with flow-level features. A malicious HTTP request and a benign one can produce similar packet counts, byte totals and timings, and the payload

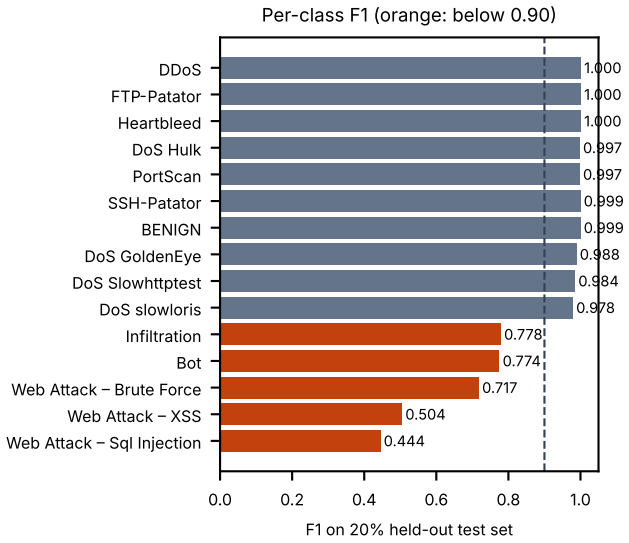


Fig. 2. Held-out per-class F1. Rare and web-attack classes are the weak region.

that distinguishes them is not part of the CICIDS2017 flow schema. Reliable detection of these attacks probably requires application-layer features, which the platform does not currently collect.

D. Operational Triage

A prediction whose confidence is below 0.75 is flagged for analyst review, and events without enough features fall back to a heuristic that is always reviewed. We have not measured the share of production-like traffic that falls below the threshold, so we report no review rate. The threshold trades analyst workload against missed detections, and it should be tuned on operational data before the platform is relied upon.

E. Verification of the Released Model Bundle

We checked the shipped bundle against its manifest. The model artifact digests match the values recorded in the manifest, and the manifest records scikit-learn 1.7.2, which matches the pinned runtime. The HMAC signature was computed with the deployment key at signing time. A full load through the application's bundle verifier with that key was not run in the verification environment, because that would have required copying the key there. That check should be run on the developer machine before release.

F. Software Verification

The backend test suite passes with 759 tests, 1 skipped and 1 deselected, on Python 3.12 with the pinned dependencies. The suite covers authentication and lockout, role-based access, project isolation, API-key storage, ingestion, incidents, evidence hashing, the ML bundle gate and its tamper checks, the incident explainer and the ML review gates. The frontend suite passes 75 tests across 12 files, and its type-check, lint and production build pass. The continuous-integration workflow runs ruff, the pytest suite, a bandit scan with medium severity as the failure threshold, and the frontend checks.

VII. Deployment and Operations

The development Compose file defines PostgreSQL, LocalStack, a migration job, the API, the worker, the frontend and a seed job. The production Compose file defines PostgreSQL, the API, the worker and the frontend, without LocalStack. The database schema is at Alembic revision 018. The Terraform definitions for AWS cover ECS, Secrets Manager, SNS alerts and AWS Backup. They have not been applied or validated in the work described here, so the platform should be described as runnable with Compose and designed, but not yet proven, for AWS.

VIII. Threat Analysis

We consider four adversaries. An external attacker may try to forge or replay ingestion keys, so keys are stored as hashes and every request is authenticated. An authenticated user may try to read another project's incidents or evidence, so access is checked per project and tested. An attacker with write access to the model directory may substitute a malicious model file; the hash and signature checks defend against this, provided the signing key is managed outside the repository.

A malicious or careless integrator may send misleading events to influence the record. LBRO cannot establish that a sender's title is true, so it keeps the sender's values separate from the model's and shows any disagreement.

A further risk concerns configuration rather than code. A deployment that falls back to a default signing secret would let anyone who knows that default forge session tokens. The release process must therefore reject missing secrets rather than substitute defaults, and any default that has ever been committed must be treated as compromised and rotated.

IX. Privacy and Ethical Considerations

Incident records and evidence can contain personal data, including IP addresses, user identifiers and payload content. The platform limits access through role-based permissions, records access in an audit log and avoids storing secrets in the repository.

It does not, however, provide a retention schedule for events or evidence, and this paper does not specify one. Any deployment that processes personal data should define retention, deletion and subject-access procedures before use. The classifier also raises an ethical point: a model trained on laboratory traffic can mislabel legitimate users, so verdicts that affect individuals should be reviewed by a person and should never trigger automatic enforcement on their own.

X. Discussion and Limitations

The evaluation is offline. CICIDS2017 was captured in a laboratory over five days, and its attacks were generated by scripted tools. Real traffic will differ in class balance, feature distribution and attacker behaviour, so the measured scores

should be treated as upper bounds on what an analyst can expect in a new environment. We have not performed a field evaluation.

Several limitations are specific to the implementation. The rate limiter keeps its counters in process memory and is therefore correct only on a single API replica. Automatic containment is off by default. It acts only when the global flag is enabled, the project has opted in, the event is critical, the verdict is not BENIGN, the confidence is at least 0.95, and the prediction is neither heuristic nor out of distribution. The effect of enabling it on a live network was not evaluated. The compliance rules have not been reviewed by counsel. Evidence storage uses PostgreSQL, and whether immutable object storage should be added remains an open design decision.

The most important lesson from the project concerns reporting. Earlier demonstration results based on synthetic data were superseded by the full-dataset evaluation reported here. A detection platform that feeds human analysts should report weak classes and evaluation caveats as prominently as strong results, and should display model disagreement with the record rather than resolve it silently.

A. Threats to Validity

Internal validity is affected by the cross-validation caveats and the earlier test-based ranking described in the methodology. External validity is limited because CICIDS2017 was generated in a controlled laboratory, and deployment environments differ in traffic mix, sensor placement and attacker sophistication. Construct validity depends on macro F1 as a proxy for analyst value. It treats every class equally, but in practice a missed web attack and a missed flood may carry very different costs, and the platform does not yet encode those costs.

B. Future Work

The most useful next step is field evaluation on operational traffic with analyst-labelled outcomes, which would allow the review threshold to be calibrated against real workload. Application-layer features for web attacks, such as request-path and parameter statistics, are the obvious addition for the weakest classes. Multi-replica deployment would require a shared store for rate limiting and event streaming. Finally, legal review of the compliance rules and a specified retention policy are prerequisites for any deployment that processes personal data.

XI. Conclusion

LBRO shows that detection, evidence preservation and breach-notification tracking can be integrated in one open platform without hiding the limits of its classifier. A histogram gradient-boosted model trained on all of CICIDS2017 reaches macro F1 0.8773 on a held-out split and 0.8485 under five-fold cross-validation. It performs strongly on flood, scan and password attacks and weakly on web attacks, Infiltration and Bot. The signed-bundle design ties the deployed model to the evaluated artifact, and the incident view surfaces disagreements between the sender and the model. Future work includes application-layer features for web attacks, field evaluation on operational traffic, calibrated review thresholds, a shared store for multi-replica deployment, removal of default secret values from deployment configuration, and legal review of the compliance rules.

References

[1] B. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153–1176, 2016, doi: 10.1109/COMST.2015.2494502.

[2] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Inf. Syst. Security Privacy (ICISSP)*, 2018, pp. 108–116, doi: 10.5220/0006639801080116.

[3] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Security Privacy*, 2010, pp. 305–316, doi: 10.1109/SP.2010.25.

[4] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[6] G. Ke *et al.*, "LightGBM: A highly efficient gradient boosting decision tree," in *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 3146–3154.

[7] P. Cichonski, T. Millar, T. Grance, and K. Scarfone, "Computer security incident handling guide," NIST Special Publication 800-61 Rev. 2, Aug. 2012, doi: 10.6028/NIST.SP.800-61r2.

[8] K. Kent, S. Chevalier, T. Grance, and H. Dang, "Guide to integrating forensic techniques into incident response," NIST Special Publication 800-86, Aug. 2006.

[9] Python Software Foundation, "pickle — Python object serialization," Python 3 Documentation. [Online]. Available: https://docs.python.org/3/library/pickle.html